

中山大學

SUN YAT-SEN UNIVERSITY

实验 0x00 实验报告

环境搭建与实验准备

姓　　名：　何亮

学　　号：　24325005

教学班号：　202520804（陈鹏飞老师班级）

专　　业：　计算机科学与技术

院　　系：　计算机学院

2025 学年第二学期

1. 尝试使用 Rust 进行编程

1. 实现 `count_down()`，`read_and_print()` 与 `file_size()` 函数，并在 `main()` 函数中进行输出测试

- 对于 `count_down()` 函数，传入一个 `seconds` 参数，使用 `while` 循环，每次 `seconds` 减 1，然后调用 `std::thread::sleep` 休眠 1s；对于 `read_and_print()` 函数，调用 `std::fs::read_to_string()` 函数访问 `file_path` 路径下的文件内容，使用 `?` 和 `io::Result<()>` 进行错误处理（路径不存在）；对于 `file_size()` 函数，调用 `std::fs::metadata(file_path)` 并使用 `Result<u64, &str>` 作为返回值。

运行结果如下：

```
linkzelda@LAPTOP-QFAEHS2I:~/hello_rust/src$ cd ~/hello_rust/src/
linkzelda@LAPTOP-QFAEHS2I:~/hello_rust/src$ cargo build
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.01s
linkzelda@LAPTOP-QFAEHS2I:~/hello_rust/src$ cargo run
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.02s
     Running `/home/linkzelda/hello_rust/target/debug/Test`
5
4
3
2
1
# This file was automatically generated by WSL. To stop automatic generation of this file, add th
e following entry to /etc/wsl.conf:
# [network]
# generateHosts = false
127.0.0.1       localhost
127.0.1.1       LAPTOP-QFAEHS2I.localdomain      LAPTOP-QFAEHS2I

# The following lines are desirable for IPv6 capable hosts
::1     ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters

Enter a file path to get its size:
./main.rs
File size: 1477 bytes
Enter a file path to get its size:
./1.rs
Error: File not Found!
Enter a file path to get its size:

linkzelda@LAPTOP-QFAEHS2I:~/hello_rust/src$
```

```rust
use std::io;

fn file_size(file_path: &str) -> Result<u64, &str> { // 尝试采用 std::Result<u64, &str> 类型作为返回值
    let metadata = std::fs::metadata(file_path).map_err(|_| "File not Found!")?;
    // 使用 ? 将错误向上转移，并使用 map_err 将错误转换为 File not Found! 进行返回
    Ok(metadata.len())
}

fn read_and_print(file_path: &str) -> io::Result<()> { // 尝试采用 std::io::Result<()> 类型作为返回值
    let contents = std::fs::read_to_string(file_path)?; // 使用 ? 将错误向上转移
    println!("{}", contents);
    Ok(())
}

fn count_down(mut seconds: u64) -> () {
    while seconds != 0 {
        println!("{}", seconds);
        seconds -= 1;
        std::thread::sleep(std::time::Duration::from_secs(1));
    }
}

fn main() {
    count_down(5);

    read_and_print("/etc/hosts").unwrap_or_else(|_| eprintln!("Error: File not found!"));
    // 使用 _ 忽略错误信息，并在错误发生时输出 File not found!

    let stdid = io::stdin();
    let mut input = String::new();
    loop {
        println!("Enter a file path to get its size:");
        stdid.read_line(&mut input).expect("Failed to read line");
        let file_path = input.trim();
        if input.trim().is_empty() {
            break;
        }
        match file_size(file_path) {
            Ok(size) => println!("File size: {} bytes", size),
            Err(err) => eprintln!("Error: {}", err),
        }
        input.clear();
    }
}
```

2. 实现一个进行字节数转换的函数

- 使用 `while` 循环进行单位换算即可，测试结果如下：

```
linkzelda@LAPTOP-QFAEHS2I:~/humanized_size/src$ cargo test
   Compiling humanized_size v0.1.0 (/home/linkzelda/humanized_size)
    Finished `test` profile [unoptimized + debuginfo] target(s) in 0.54s
     Running unittests src/main.rs (/home/linkzelda/humanized_size/target/debug/deps/humanized_size-f6
2a96342b43ef15)

running 1 test
test tests::test_humanized_size ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s
```

```rust
fn humanized_size(size: u64) -> (f64, &'static str) {
    let units = ["B", "KiB", "MiB", "GiB"];
    let mut size = size as f64;
    let mut unit_index = 0;

    while size >= 1024.0 && unit_index < units.len() - 1 {
        // 进行单位换算，同时限制换算的最大单位为 GiB
        size /= 1024.0;
        unit_index += 1;
    }

    (size, units[unit_index])
}
```

补全后的测试代码如下：

```rust
#[test]
    fn test_humanized_size() {
        let byte_size = 1554056;
        let (size, unit) = humanized_size(byte_size);
        assert_eq!("Size :  1.4821 MiB", format!("Size :  {:.4} {}", size,
unit));
    }
```

3. 在终端输出彩色的文字

- 这里采用的是 ANSI 转义序列实现：ANSI 转义序列以 ESC 字符（对应的 ASCII 码为 `\x1B`）开头，然后按照格式设置文字颜色（绿色，黄色，红色分别对应 32, 33, 31），然后设置下划线（对应的参数为 1），然后根据命令行的宽度设置空格 padding 进行居中处理。

代码运行效果如下：

```
linkzelda@LAPTOP-QFAEHS2I:~/colorful_terminal/src$ cargo build
   Compiling colorful_terminal v0.1.0 (/home/linkzelda/colorful_terminal)
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.35s
linkzelda@LAPTOP-QFAEHS2I:~/colorful_terminal/src$ cargo run
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.01s
     Running `/home/linkzelda/colorful_terminal/target/debug/colorful_terminal`
 INFO: Hello, World!
 WARNING: I'm a teapot!
                          ERROR: KERNEL PANIC!!!
linkzelda@LAPTOP-QFAEHS2I:~/colorful_terminal/src$
```

```rust
fn main() {
    print!("\x1b[32mINFO: \x1b[0mHello, World!\n");

    print!("\x1b[33;1;4mWARNING\x1b[0m: \x1b[33;1mI'm a teapot!\x1b[0m\n");

    let message = "ERROR: KERNEL PANIC!!!";
    let term_width: usize = 80; // 指定宽度

    let padding = (term_width.saturating_sub(message.len())) / 2;
    let spaces = " ".repeat(padding);

    print!("{}", spaces);
    print!("\x1b[31;1m{}\x1b[0m\n", message);
}
```

4. 使用 `enum` 对类型实现同一化

• 使用 `enum` 对矩形和圆两种类型进行同一化，然后使用 `match` 语句根据类型计算面积即可。

测试结果如下：

```
linkzelda@LAPTOP-QFAEHS2I:~/shape$ cd ~/shape/src/
linkzelda@LAPTOP-QFAEHS2I:~/shape/src$ cargo test
    Finished `test` profile [unoptimized + debuginfo] target(s) in 0.12s
     Running unittests src/main.rs (/home/linkzelda/shape/target/debug/deps/Shape-64c3c8eee5d657f5)

running 1 test
test tests::test_area ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s
```

```rust
use std::f64::consts::PI;

enum Shape {
    Rectangle { width: f64, height: f64 }, // 对应矩形
    Circle { radius: f64 }, // 对应圆
}

impl Shape {
    fn area(&self) -> f64 {
        match self { // 用 match 来判断其类型，并计算面积
            Shape::Rectangle { width, height } => width * height,
            Shape::Circle { radius } => PI * radius * radius,
        }
    }
}
```

5. 实现一个元组结构体 `UniqueId(u16)`

- 使用一个 `AtomicU16` 类型的 `static` 变量来对每个新声明的 `UniqueId` 类型变量编号，使它们不相等即可通过测试。

测试结果如下：

```
linkzelda@LAPTOP-QFAEHS2I:~/lab-repo$ cd ~/UniqueId/src/
linkzelda@LAPTOP-QFAEHS2I:~/UniqueId/src$ cargo test
    Finished `test` profile [unoptimized + debuginfo] target(s) in 0.05s
     Running unittests src/main.rs (/home/linkzelda/UniqueId/target/debug/deps/UniqueId-760fd6cca4ce6d
9d)

running 1 test
test tests::test_unique_id ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s
```

```rust
use std::sync::atomic::{AtomicU16, Ordering};

#[derive(Debug, PartialEq, Eq)]
struct UniqueId(u16);

impl UniqueId {
    fn new() -> Self {
        static COUNTER: AtomicU16 = AtomicU16::new(0);
        // 创建 AtomicU16 类型的变量使其在多个线程之间也能生成不同的值
        let id = COUNTER.fetch_add(1, Ordering::Relaxed);
        // 返回 COUNTER 的值之后 +1
        UniqueId(id)
    }
}
```

2. 运行第一个 UEFI 程序

按照实验文档中的说明配置好环境之后，将 `crates/boot/src/main.rs` 下的代码中的 `/* FIXME */` 改成 `24325005`（我的学号），然后运行即可看到对应的结果。

```
linkzelda@LAPTOP-QFAEHS2I:~/YatSenOS-Tutorial-Volume-2/src/0x00$ make run
cd crates/boot && cargo build --release
    Finished `release` profile [optimized] target(s) in 0.25s
cp target/x86_64-unknown-uefi/release/ysos_boot.efi esp/EFI/BOOT/BOOTX64.EFI
BdsDxe: failed to load Boot0001 "UEFI QEMU DVD-ROM QM00003 " from PciRoot(0x0)/Pci(0x1,0x1)/Ata(Secondary,
Master,0x0): Not Found
BdsDxe: loading Boot0002 "UEFI QEMU HARDDISK QM00001 " from PciRoot(0x0)/Pci(0x1,0x1)/Ata(Primary,Master,0
x0)
BdsDxe: starting Boot0002 "UEFI QEMU HARDDISK QM00001 " from PciRoot(0x0)/Pci(0x1,0x1)/Ata(Primary,Master,
0x0)
[ INFO]: crates/boot/src/main.rs@019: Hello World from UEFI bootloader! @ 24325005
[ INFO]: crates/boot/src/main.rs@019: Hello World from UEFI bootloader! @ 24325005
[ INFO]: crates/boot/src/main.rs@019: Hello World from UEFI bootloader! @ 24325005
[ INFO]: crates/boot/src/main.rs@019: Hello World from UEFI bootloader! @ 24325005
```

3. 思考题

1. 了解现代操作系统（Windows）的启动过程，UEFI 和 Legacy（BIOS）的区别是什么?

- 现代操作系统（以 Windows 为例）的启动过程：主要以 UEFI 模式进行启动。当计算机开机时，UEFI 固件会首先进行自检（检测硬件是否正常工作），然后加载并执行位于 ESP 分区中的引导程序。引导程序会加载 Windows 内核和一些必要的驱动程序，最后将控制权交给操作系统内核，完成启动过程。
- UEFI 相比 BIOS 提供了更为先进的接口和功能。UEFI 支持更大的硬盘（BIOS 仅支持约 2TB 的硬盘）、拥有更快的启动速度以及更好的安全性和图形界面，而 BIOS 则较慢，安全性较低，功能较为有限且不支持 32 位系统和 64 位系统。

2. 尝试解释 Makefile 中的命令做了哪些事情？或许你可以参考下列命令来得到更易读的解释：

```
python ysos.py run --dry-run
```

运行上述命令后会观察到如下输出

```
linkzelda@LAPTOP-QFAEHS2I:~/YatSenOS-Tutorial-Volume-2/src/0x00$ python3 ysos.py
run --dry-run
[+] Building: bootloader...
[?] Executing: /home/linkzelda/.cargo/bin/cargo build --release in /home/
linkzelda/YatSenOS-Tutorial-Volume-2/src/0x00/crates/boot
[?] Would copy: /home/linkzelda/YatSenOS-Tutorial-Volume-2/src/0x00/target/
x86_64-unknown-uefi/release/ysos_boot.efi -> /home/linkzelda/YatSenOS-Tutorial-
Volume-2/src/0x00/esp/EFI/BOOT/BOOTX64.EFI
[?] Executing: /usr/bin/qemu-system-x86_64 -bios assets/OVMF.fd -net none -
nographic -m 96M -drive format=raw,file=fat:esp -snapshot
```

- 首先，脚本通过运行 cargo 将 rust 源码编译生成为可以在 UEFI 环境下运行的 .EFI 文件，生成的文件会保存在 `./target/x86_64-unknown-uefi/release/ysos_boot.efi` 路径下。

- 随后, 脚本将生成的 .EFI 文件复制到 `./esp/EFI/BOOT/BOOTX64.EFI` 路径下（该路径是 X86_64 架构下 UEFI 固件在启动时，查找可引导程序的默认路径）。
- 随后启动硬件模拟器 QEMU 用来模拟 UEFI 环境，其中的参数分别禁用了网络设备，让输出直接输出到命令行窗口，分配了 96M 的内存，将 `./eps` 目录模拟为一个 FAT 格式的磁盘供其启动使用，同时启动 `snapshot` 模式，使其不会实际修改 `./esp` 目录下的内容。

3. 利用 cargo 的包管理和 docs.rs 的文档，我们可以很方便的使用第三方库。这些库的源代码在哪里？它们是什么时候被编译的?
- 这些库的源代码通常放在 `.cargo/registry/src/`，若该路径下不存在所需要的包，则会从 `cargo.io` 上下载对应的包然后放在该路径下。
- 它们是在项目构建的时候被编译的，cargo 首先会分析所有包之间的依赖关系，然后按照拓扑序进行逐层编译。

4. 为什么我们需要使用 entry 而不是直接使用 main 函数作为程序的入口?
- 因为我们需要编写出来的程序可以在 UEFI 环境下运行，该环境下没有操作系统提供的标准库和 Interpreter，不能以 main 函数作为程序的入口函数。因此需要用 `#[entry]` 属性来指明程序的入口点。

5. 结合本实验代码：为什么忙等待（nop 循环）会影响系统可观测性与能耗？若改为基于 UEFI 定时事件的等待，你预期日志输出的时间稳定性会有什么变化?
- 首先，因为忙等待会向汇编代码中加入大量 `nop` 空语句，使得系统中的 CPU 要执行更多的语句，因此会影响能耗；同时，由于 CPU 在执行 `nop` 循环中所有 `nop` 的语句的总时间是不确定的，可能会影响日志输出的时间稳定性，以及 `nop` 语句长期占用 CPU 可能会让中断响应和调试接口产生延迟，因此 nop 循环会影响系统的可观测性和能耗。
- 如果改为 UEFI 定时时间的等待，我认为日志输出的时间稳定性会大大提升，因为者不依赖于 CPU 执行 `nop` 空语句的效率，而是通过内部硬件的时钟实现的，因此相邻日志之间的时间间隔会更为稳定准确。

6. 在 `no_std` 环境下，为什么“可观测性（日志/断言/错误路径）”往往比业务逻辑更难补齐?
- 因为 `std` 库中提供了大量封装好的宏，如 `eprintln!`, `assert!` 等，可以较为方便地维护可观测性；但是在 `no_std` 环境下，需要自行从底层构建对应的功能，因此更难补齐。

7. 如果把本实验的 UEFI 程序迁移到另一台固件实现不同的机器上，你认为最先出现兼容性问题的环节会是什么？为什么?
- 我认为是 nop 循环可能最先出现兼容性问题。因为在不同固件实现的机器上，使用指令集的类型不同，同时查阅资料有些固件实现存在“看门狗定时器”机制，以及不同固件实现中的 CPU 频率不同，因此 nop 循环很可能出现兼容性问题（在不同机器上的表现不同）。

4. 加分项

1. 基于控制行颜色的 Rust 编程题目，参考 log crate 的文档，为不同的日志级别输出不同的颜色效果，并进行测试输出。

参考了 `log crate` 文档，自定义了 `ColoredLogger` 结构体实现对不同级别的日志输出不同的颜色效果。

```rust
use log::{Level, LevelFilter, Log, Metadata, Record};
use std::time::SystemTime;


struct ColoredLogger;

impl Log for ColoredLogger {
    fn enabled(&self, metadata: &Metadata) -> bool {
        metadata.level() <= Level::Trace
    }

    fn log(&self, record: &Record) {
        if !self.enabled(record.metadata()) {
            return;
        }

        // 根据日志级别选择 ANSI 颜色代码
        let (_color_code, reset_code) = ("\x1b[0m", "\x1b[0m"); // 默认无色
        let level_style = match record.level() {
            Level::Error => "\x1b[31;1m", // 红色粗体
            Level::Warn  => "\x1b[33m",   // 黄色
            Level::Info  => "\x1b[32m",   // 绿色
            Level::Debug => "\x1b[34m",   // 蓝色
            Level::Trace => "\x1b[90m",   // 灰色
        };

        // 获取当前时间
        let _now = SystemTime::now()
            .duration_since(SystemTime::UNIX_EPOCH)
            .map(|d| d.as_secs())
            .unwrap_or(0);

        // 格式化输出：时间 [级别] 目标 - 消息
        println!(
            "{}{:5} {} - {}{}",
            level_style,
            record.level(),
            record.target(),
            record.args(),
            reset_code
        );
    }

    fn flush(&self) {}
}
```

代码测试运行效果如下：

```
linkzelda@LAPTOP-QFAEHS2I:~/colorful_terminal$ cargo run
   Compiling colorful_terminal v0.1.0 (/home/linkzelda/colorful_terminal)
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 1.34s
     Running `target/debug/colorful_terminal`
ERROR colorful_terminal - Kernel panic
WARN  colorful_terminal - I'm a teapot!
INFO  colorful_terminal - Hello, world!
DEBUG colorful_terminal - Debugging info: $rdi = &boot_loader
TRACE colorful_terminal - Entering function Test, args: (123, "OVO")
```

2. 实现一个简单的 shell 程序。

基于第一个 rust 实验，我采用标准库中的方法来实现一个 shell，其中 `cd` 使用 `std::env::set_current_dir` 方法实现；`cat` 通过使用已经实现的 `read_and_print` 方法来实现；`ls` 通过 `std::fs::read_dir` 方法结合其返回类型自带的 `.file_name()`，`.metadata().len()` 与 `metadata().modified()` 等成员函数实现，获取当前路径下的文件名，文件大小和修改时间，运行效果如下：

```
linkzelda@LAPTOP-QFAEHS2I:~/hello_rust$ cargo run
66 | fn file_size(file_path: &str) -> Result<u64, &str> {
   |    ^^^^^^^^^

warning: `Test` (bin "Test") generated 2 warnings
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.31s
     Running `target/debug/Test`
/home/linkzelda/hello_rust $ ls
Name                                          Size  Modified
src                                           4096  SystemTime { tv_sec: 1772515729, tv_nsec: 364706083 }
Cargo.toml                                      75  SystemTime { tv_sec: 1772514176, tv_nsec: 316638769 }
.git                                          4096  SystemTime { tv_sec: 1772514177, tv_nsec: 48909879 }
target                                        4096  SystemTime { tv_sec: 1773041539, tv_nsec: 694652459 }
Cargo.lock                                     148  SystemTime { tv_sec: 1772514250, tv_nsec: 702446490 }
.gitignore                                       8  SystemTime { tv_sec: 1772514176, tv_nsec: 316638769 }
/home/linkzelda/hello_rust $ cd ./../../linkzelda/hello_rust/src/../../src
/home/linkzelda/hello_rust/src $ ls
Name                                          Size  Modified
main.rs                                       5286  SystemTime { tv_sec: 1773798252, tv_nsec: 713496018 }
/home/linkzelda/hello_rust/src $ cat main.rs
// use std::io;

// fn file_size(file_path: &str) -> Result<u64, &str> { // 尝试采用 std::Result<u64, &str> 类型作为返回值
//     let metadata = std::fs::metadata(file_path).map_err(|_| "File not Found!")?;
//     // 使用 ? 将错误向上转移，并使用 map_err 将错误转换为 File not Found! 进行返回
//     Ok(metadata.len())
// }
```

3. 尝试使用线程模型，基于 `UniqueId` 的任务。

• 尝试证明 `static mut` 变量在多线程下的不安全（可能获得相同的 UniqueId）

我们将 `UniqueId` 结构体中 `COUNTER` 设置为 `static mut`，然后创建 100 个线程，每个线程声明 100 个 `UniqueId` 类型的变量，然后收集所有的声明的变量的 ID，观察是否会出现重复。测试代码如下：

```rust
#[test]
    fn test_unique_id_multi_thread() {
        let thread_count = 100;
        let ids_per_thread = 100;
        let expected_total = thread_count * ids_per_thread;

        let mut handles = vec![];
        for _ in 0..thread_count {
            handles.push(thread::spawn(move || {
                let mut local_ids = vec![];
                for _ in 0..ids_per_thread {
                    local_ids.push(UniqueId::new().0);
                }
                local_ids
            }));
        }

        let mut all_ids = vec![];
        for h in handles {
            all_ids.extend(h.join().unwrap());
        }

        assert_eq!(all_ids.len(), expected_total);

        all_ids.sort();
        let duplicates: Vec<_> = all_ids.iter()
            .enumerate()
            .filter_map(|(i, &val)| if i > 0 && val == all_ids[i-1] { Some(val) }
else { None })
            .collect();
        assert!(duplicates.is_empty(), "dupilicate ID: {:?}", duplicates);
    }
```

```
   Compiling UniqueId v0.1.0 (/home/linkzelda/UniqueId)
warning: unused import: `std::thread`
  --> src/main.rs:28:5
   |
28 | use std::thread; …
b/src/rust/library/core/src/ops/function.rs:250:5
   5: core::ops::function::FnOnce::call_once
             at /rustc/01f6ddf7588f42ae2d7eb0a2f21d44e8e96674cf/library/core/src/ops/function
.rs:250:5
note: Some details are omitted, run with `RUST_BACKTRACE=full` for a verbose backtrace.
test tests::test_unique_id_multi_thread ... FAILED

failures:

failures:
    tests::test_unique_id_multi_thread

test result: FAILED. 0 passed; 1 failed; 0 ignored; 0 measured; 1 filtered out; finished in 0
.09s

error: test failed, to rerun pass `-p UniqueId --bin UniqueId`

 *  The terminal process "cargo 'test', '--package', 'UniqueId', '--bin', 'UniqueId', '--', '
tests::test_unique_id_multi_thread', '--exact', '--nocapture'" terminated with exit code: 101
.
 *  Terminal will be reused by tasks, press any key to close it.
```

我们经过测试，出现了重复的情况，说明在多线程下，使用 `static mut` 是不安全的。

- 尝试验证 `AtomicU16` 来实现 `UniqueId` 时的正确性

对于 `AtomicU16` 类型，其自增并返回值的成员函数属于原子操作（即指令不可再分，不会被编译器进行指令重排优化），因此在多线程下是安全的。

```
    Finished `test` profile [unoptimized + debuginfo] target(s) in 0.00s
     Running unittests src/main.rs (target/debug/deps/UniqueId-760fd6cca4ce6d9d)

running 1 test
test tests::test_unique_id_multi_thread ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 1 filtered out; finished in 0.02s

 *  Terminal will be reused by tasks, press any key to close it.
```

我们使用前面的测试进行验证，通过验证，我们证明了 `AtomicU16` 在多线程下的安全性。